REGISTERED BY AUSTRALIA POST PUBLICATION No. TBH 0917 CATEGORY "B"

ISSUE NO.	ANNUAL SUBSCRIPTION	DATE
3 - 2	AUSTRALIA ..... $20.00 OVERSEAS ..... $25.00 OVERSEAS AIRMAIL ... $30.00 YEAR BOOK 83/84 .... $15.00	NOV - 1985

CONTENTS

NEWSLETTER CORRESPONDENCE

S.A.U.G.,
P.O. BOX 191,
LAUNCESTON SOUTH,
TASMANIA, 7249.

LIBRARY CORRESPONDENCE

S.A.U.G. LIBRARY,
1 CONRAD AVENUE,
GEORGE TOWN,
TASMANIA, 7253.

(003) 822919

Exploring Basic Pt-14

by L.A. Dunning

This part deals with disk drives and how to cope with them. SV Disk drives seem to come in two varieties, those that break down and those that don't. Assuming you have a system that works, the following comments should be of use.

DISK SCANNER

Before reading the rest of this article, I would suggest you type in the listing in this issue. This will enable you look at disks while reading the article. One word of warning when using the program, it always assumes that there is a disk inside the disk drive to scan. If there isn't, it will hang until you place a formatted disk inside the drive. A handy advantage of the program, one I discovered by accident, is that it can be used to read CP/M disks as well. I will talk about this later.

FORMATS

There are two ways to format a disk so that it can be used in the disk drive. The first is in BASIC and the second is in CP/M. The same CP/M program is used to format both. The difference is that on a CP/M disk, the system file is the CP/M disk operating system; on a BASIC disk it is the BASIC disk operating system. Also, a BASIC disk has a seperate directory track. Each formatted disk is divided into 40 tracks of 17 sectors each and each sector contains 256 bytes of information. Thus, each disk can contain a maximum of 174,080 bytes. Of these, 4 tracks are reserved so this figure is reduced by 17,408 bytes to 156,672 bytes. The tracks are numbered from 0 to 39, with 0 being the innermost track and 39 being the outermost. The choice of number for sectors is really arbitrary however each track is set to the same sequence from 1 to 17. Tracks 0 to 2 contain the system file that is read when the system is booted up. Track 20 is the directory track for a basic disk.

Before going any further, format a new disk in the normal manner and setup the directory track as usual, but do not save any files on the disk yet. Load the DISK SCANNER program into memory and insert the newly formatted disk into drive 1, then run the program. Track 20 is used to record three tables that are used to reference files on the disk. These are the Directory table (DIR), the Disk Allocation Table (DAT) and the File Allocation Table (FAT).

DIRECTORY TABLE

The DIR starts at sector 1 and allowance is made to sector 13. On a normal disk you wont use more than the first 6 sectors for the DIR. On double sided drives with modified operating systems I can only make an educated guess (not possessing one myself) however 13 sectors would give enough room for all possible files plus more. If you are running the program you will see that the intial sector is set to track 20, sector 1, which is the start of the DIR. On the new disk this set to a block of FFH bytes. There are no files yet.

Each entry in the DIR consists of 16 bytes. The first 9 are used for the file name which is 6 characters plus and extension of 3 characters. Shorter file names are padded with blanks. The first character has a special purpose. If it is FFH, it indicates the end of the directory and only entries before this will be listed by FILES. If it is 00H, it indicates a deleted file and this is not listed either by FILES. Such an entry is used for new files before creating a new entry. The 10th byte is the attribute byte and is used to indicate the type of file. It is easier to note this byte in octal instead of hex. The following values indicate the types present:

000	ASC File or data file
001	Machine Code file
200	Binary File

In addition the following values indicate extra attributes:

010	Read after Write	"R"
020	Write Protect	"P"
040	Screen Dump	

A binary file is the normal method that a BASIC program is saved to disk. It is a direct dump from what is in memory. All screen dumps are also binary files. An ASC file is the method used to save a BASIC program using the "A" option. It prints to disk in the same way a program is LISTed to the screen. The advantage of ASCII files is that they are easier to manipulate, but use more sectors than a binary file. Data files using PRINT# and PUT# are also in ASCII format. A Machine code file is really just a binary file and represents a dump from memory. The difference is that it is dumped directly back to memory. The 11th byte is used to point to the first entry in the FAT for the file. Bytes 12 to 16 are "reserved for future use" which means they are untouched by a normal operating system and are normally left at FFH.

DISK ALLOCATION TABLE

This is sector 14 on track 20. Originally this is blanked to 00H. This table has only two entries. The first byte is used for disk attributes, for Read after Write and Write Protect options. It is set to the same values as the attribute byte in a DIR entry. The difference is that values in the DIR entry apply only to that file, whereas a value here applies to the entire disk. The second entry is the string executed by the IPL command. This is normally no more than 15 bytes long. Any shorter strings are padded by 00H's. The rest of this sector is not used.

FILE ALLOCATION TABLE

This table is repeated in sectors 15, 16 & 17 on track 20. Normally only the first 41 bytes are used. The first 40 bytes form a 'linked list' that indicates which tracks are used by which files. Each entry corresponds to one track on the disk, the first being track 0 and the last being track 39. Originally each entry on this table has one of two values. An FFH indicates a free track, a FEH indicates a reserved track. At the start, only tracks 0, 1, 2 & 20 are reserved. A reserved track is not used to store files on. As a file is created, the system decides on which track to start placing that file. Normally the system will pick unused tracks that are closest to the Directory Tract and then work outwards. The 11th byte in a DIR entry will point to the byte on the FAT corresponding to the first track used. If that

entry is a number from 0 to 39, it points to the next track and entry used. The entry for this next track will then point to the next track/entry and so on, until the file ends. If a track is the last one used for a file, its entry will be the number of sectors used (from 1 to 17) ORed with 300 octal. While a file is deleted, the first byte on the DIR entry is set to 00H and the entries on the FAT are set to FFH indicating a free track. The Attribute and pointer bytes in the DIR entry are untouched. The file itself is left on the tracks until overwritten by another file. If you KILL a file by mistake, you can use DISK SCANNER or a similar utility to restore the file, by altering entries on the DIR and FAT. All that is required is a knowledge of where the file starts and what it should look like. This will take practice. The final byte in the FAT is set to 00H and indicates the end of the table. Presumerably on an 80 track disk, the FAT is 80 entries long followed by a zero byte. You will notice that the FAT is repeated three times. I believe this is to reduce the chances of losing files due to hardware or software bugs. If you wipe sector 15, copy sectors 16 or 17 to it. The system also periodically copies the DIR, so you may find a copy of sector 1 in sector 2. As new entries are added, deleted or changed, the copy becomes obsolete until once again copied.

FILES

Each file is saved to disk on a sector by sector basis. Each 256 bytes of the file is dumped to a sector of a track, starting at sector 1 and continuing until sector 17. When all sectors of a track are used, a new track is picked and the process repeated. This is made possible by first filling a buffer and then writting the contents to the disk. This is not done until either the buffer is full, or the file closed. Each file starts at sector 1 and if less than 17 sectors are used, the remaining sectors on that track are left unused except for expansion of that file. The first byte of a screen dump file is used to indicate the type of screen saved. It will be either 0, 1 or 2. The first six bytes of a machine code file are used to indicate information about the dump. The first two indicate the start of the dump, the second two indicate the length of the dump and the last two indicate the entry point of the code. Each pair comes in the usual Low byte/High byte manner of integer values.

The use of a linked list in the FAT enables a file to be on any available track of a disk instead of consecutive tracks. If you are using a disk as a data disk and do not need the system file on it, use DISK SCANNER or equivilant to change the entries for tracks 0 to 2 on the FAT to FFH. This will give you an extra 51 sectors of storage.

SYSTEM FILES

Tracks 0 to 2 are the system file that is used when the system is booted. If you look at track 0 you will see that the last 128 bytes of every sector is identical. When the computer is booted, it scans this track and loads its instructions from it, dumping the rest of the file into memory. You might use DISK SCANNER to change the message given when first displayed. You will find this on sectors 0/5 & 0/6.

As the program also reads CP/M disks, you can alter both the directory entries and function key definitions given on Track 0. These are in sectors 3/1 onwards, and 0/2 to 0/4 respectfully. Be very careful altering the first of these, or you might lose some files. It does not conform to the description given above!

DISK SCANNER

By now you have either saved some files to check the above, or you want to do so now. I would suggest first saving DISK SCANNER in normal format, then saving it in ASC. Then save the screen and dump a segment of memory to disk using BSAVE. Then delete a file. After each change, check the DIR and FAT to see how they are altered. In case you were wondering, diagram 1 gives a listing of the MCR used in the program. It works by dumping buffer #0 to the screen. Buffer #0 is used by the system for most disk input/output. The subroutine in lines 400-410 swap two NOP's for the JR DUMP at 0016, thereby altering the MCR flow. Lines 260-270 alter values in the MCR when it is being poked into buffer #1. You can place your own values here to change the origin and width of the dump. The reason it was done in this manner was so that you could also use the routine for your own purposes, namely reading a segment of memory. All that is needed is the starting byte and this is given by the argument in the J=USR() statements. Subroutines 700-710 read and restore the function keys to whatever values they were before the program. This uses a string array B(A,N) where A is the number of the function key and N is the number of definitions needed. You can read and restore any number of definitions just by increasing N. A dummy string ,BD, is required in line 700 to prevent an error from occuring.

NEXT TIME

Continuing on the subject of disks, I'll show how to get effective use of them from basic.

Disk Scanner

Except for line 930 this is a standard SVI BASIC program and can be entered using the INPUT program if you leave out line 930 until you have finished typing in the rest of the program. Line 930 makes special use of the Cursor Arrow Keys and you will have to follow suit. Set up your function keys so that F1 to F4 equate to the four graphics arrow characters as shown in your handbook to be ASCII characters 212, 213, 214, and 215. When entering line 930 where an arrow character is called for simply press the appropriate function key as you would an ordinary keyboard key. 'old' members of the group, and those with the yearbook could refer to the short article in Newsletter 1 - 11 for August,84. For new members and those too tired to leaf through the past issues I offer this ray of hope :-

```
KEY1,CHR$(212)
KEY2,CHR$(213)
KEY3,CHR$(214)
KEY4,CHR$(215)
```

DISK SCANNER

by : L.A. Dunning.

```
AG      10 REM DISK SCANNER
FM      19 REM Initialisation
DJ      20 WIDTH39:MAXFILES=2:CLEAR1000:DEFINTA-Z:DEFSTR B:DIM DX(15),DY(15),B(6)
        ,BK(10,1),I(1),D(1),T(1),S(1),BD:XA=VARPTR(#0)+9:XB=VARPTR(#1)+9:B="DI
        SK:":PX=14:PY=2:D=1:T=20:S=1:DT=1:MT=0:AA=0:FORA=0TO8:READ DX(A),DY(A)
        :NEXT:GOSUB210
FO      30 BS=CHR$(8):BC=CHR$(27):DEF FNBH(X)=STRING$(2-LEN(HEX$(X)),"0")+HEX$(X)
        :DEF FNBO(X)=STRING$(3-LEN(OCT$(X)),"0")+OCT$(X):VX=4:VY=PY+6
HP      40 N=0:GOSUB700:GOSUB930:KEY1," HEX ":ONSTOPGOSUB3000:STOP ON:KEY ON:DEFU
        SR=VARPTR(#1)+9:GOSUB730:FIELD#0,128ASB(0),128ASB(1)
HP      50 FORA=2TO6:READB(A):NEXT
AP      99 REM Viewing Routines
DA     100 D$=DSKI$(D,T,S)
DH     110 I$=INKEY$:J=USR(XA)
IJ     120 LOCATEPX+1,PY+18,0:PRINTUSING"D:##  T:##  S:##";D,T,S:GOSUB200:IFD1=0A
        NDD2=0GOTO120
DH     130 IFD2THEN D=3-D
BB     140 T=T+DY(D1):T=T-((T<0)-(T>39))*40:S=S+DX(D1):S=S-((S<1)-(S>17))*17
AC     150 GOTO 100
IA     199 REM General Routines #1
IJ     200 FORTL=1TO10:NEXT:D1=STICK(0)ORSTICK(1):D2=STRIG(0)ORSTRIG(1):RETURN
DD     210 XX=XB:CC=0
FF     220 READ BV:L=LEN(BV)
DN     230 IF L=3 GOTO 250 ELSE VV=VAL("&H"+BV)
CA     240 POKE XX+CC,VV:CC=CC+1:GOTO220
AF     250 IF BV="END"THENRETURN
IH     260 IF BV="wit"THENVV=16
EC     270 IF BV="vid"THENVV=40+PX+PY*40+2
AD     280 GOTO240
ID     399 REM General Routines#2
BC     400 IFDT=1 THEN POKEXX+22,0:POKEXX+23,0 ELSE POKEXX+22,&H18:POKEXX+23,&HF
BJ     410 DT=1-DT:KEY1," "+MID$("ASCHEX",1+DT*3,3):J=USR(XA):RETURN
BL     430 BEEP:GOSUB 740:CX=0:CY=0:LOCATEVX-1,VY-1:PRINT"Hex:Oct":LOCATEVX,VY+2:
        PRINT"Mod:":RETURN440
AL     440 I$=INKEY$:LOCATEPX+CX+1,PY+CY+1,1:GOSUB200
EA     450 GOSUB650:LOCATEVX,VY,0:PRINTFNBH(VV)":"FNBO(VV):GOTO440
GN     499 REM Modification Routines
CM     510 LOCATEPX+CX+1,PY+CY+1:PRINTBC"p"MID$("HA",AA+1,1)BC"q":VV$=""
HM     520 LOCATEVX+4,VY+2,1:PRINTVV$;:BI=INKEY$:AC=((PEEK(&HFD7D)AND16)=0):IFACT
        HEN AA=1-AA:GOTO510
CF     530 IFBI=""GOTO520
FI     540 IFBI=CHR$(27)GOTO640
AF     550 ON AA GOTO 630
DK     560 IFINSTR("0123456789ABCDEFabcdef",BI)ANDLEN(VV$)<2THENVV$=VV$+BI
ED     570 IF BI=BSANDLEN(VV$)>0THENVV$=LEFT$(VV$,LEN(VV$)-1)
BD     580 IFBI=" "ORBI=BS THEN IFLEN(VV$)<1THENVV$=VV$+HEX$(VV\16) ELSE GOSUB60
        :D1=3:GOSUB650:GOTO510
CM     590 IFBI<>CHR$(13)GOTO520
OJ     600 LL=LEN(VV$):NV=VAL("&h"+VV$):IFLL=2THENVV=NV
CE     610 IFLL=1THENVV=VVAND15:VV=VV+NV*16
AI     620 POKEXA+PP,VV:GOTO640
BO     630 VV=ASC(BI):GOSUB620:D1=3:GOSUB650:GOTO510
```

```
BJ    640 J=USR(XA):RETURN
CL    650 CY=CY+DY(D1):CY=CY-((CY<0)-(CY>15))*16:CX=CX+DX(D1):CX=CX-((CX<0)-(CX
       15))*16:PP=CX+CY*16:VV=PEEK(XA+PP):RETURN
IN    660 DSKO$D,T,S:D$=DSKI$(D,T,S):J=USR(XA):RETURN
AE    670 BEEP:GOSUB730:RETURN100
HP    699 REM General Routines#3
IM    700 JK=VARPTR(BD):POKEJK,16:POKEJK+2,250:FORA=1TO10:POKEJK+1,14+A*16:BK(A
       N)=BD:BK(A,N)=LEFT$(BK(A,N),INSTR(BK(A,N),CHR$(0))-1):NEXT:RETURN
FO    710 FORK=1TO10:KEYK,"":KEYK,BK(K,N):NEXT:RETURN
HB    730 MM=0:GOSUB760:LOCATE3,3:PRINT"Track  ":LOCATE3,4:PRINT"Sector    ":KE
       2,"<COPY>":KEY3,"<MOD.>":KEY4,"<HELP>":KEY5,"<FREE>":ONKEYGOSUB400,800
       ,430,1000,830:RETURN
DK    740 MM=1:GOSUB760:LOCATE3,3:PRINT"Up/Down":LOCATE3,4:PRINT"Left/Right":KE
       2,"<COPY>":KEY3,"<VIEW>":KEY4,"<ALT.>":KEY5,"<BURN>":ONKEYGOSUB400,800
       ,670,510,660:RETURN
DA    750 MM=2:GOSUB760:LOCATE3,3:PRINT"        ":LOCATE3,4:PRINT"          ":KE
       3,"<MOD.>":KEY2,"<VIEW>":KEY4,"<SLCT>":KEY5,"<DUMP>":ONKEYGOSUB400,670
       ,430,860,820:RETURN
BJ    760 LOCATE0,0:PRINTMID$("VIEWMOD.COPY",1+MM*4,4)" MODE":FORA=5TO20:LOCATE
       ,A:PRINTSPC(12):LOCATEVX,VY:PRINT"  ";:NEXT:RETURN
ID    799 REM Copy Routines
AK    800 BEEP:GOSUB750:I(0)=0:I(1)=0:RETURN810
BF    810 I$=INKEY$:GOTO 810
LA    820 IF I(0)ANDI(1) THEN D$=DSKI$(D(0),T(0),S(0)):DSKO$D(1),T(1),S(1):D=D(
       ):T=T(1):S=S(1):J=USR(XA):RETURN ELSE RETURN
AP    830 T=20:S=15:RETURN100
DG    840 PRINTBS;BS;:LINEINPUTVV$:VV=ABS(VAL(LEFT$(VV$,2))):IFINSTR(VV$,"*")TH
       NI(E)=0:E=2:RETURN920 ELSE RETURN
BB    860 FORE=0TO1
AI    870 LOCATE0,6+E*6,1:PRINTMID$("FROMTO  ",1+E*4,4)":"
AE    880 PRINTUSING"  DISK:# ";D(E);:GOSUB840:D(E)=VV
AG    890 PRINTUSING" TRACK:##";T(E);:GOSUB840:T(E)=VV
FF    900 PRINTUSING"SECTOR:##";S(E);:GOSUB840:S(E)=VV
NK    910 I(E)=1+(D(E)<1ORD(E)>2ORT(E)>39ORS(E)<1ORS(E)>39):IFI(E)=0GOTO870 ELS
       D$=DSKI$(D(E),T(E),S(E)):J=USR(XA)
AI    920 NEXT:LOCATE,,0:RETURN
DK   1000 CLS:PRINT" DISK SCANNER INSTRUCTIONS":PRINT:PRINT" This program allow
       the user to view, modify and copy disk sectors on an SV  902 disk dri
       ve.":PRINT
HG   1010 PRINT" There are 3 modes to use, press":PRINT"<0> to return to VIEW":
       RINT"<1> for VIEW information":PRINT"<2> for MOD.":PRINT"<3> for COPY"
BB   1020 PRINT:PRINT" ^STOP to exit program":PRINT:PRINT" Each sector is displ
       yed in a 16x16  grid in either HEX or ASC display. HEX dumps each val
       ue as a straight patten"
BL   1030 PRINT"value. ASC shows ASCII characters in  the proper manner and in
       erts values 0 to 31."
CD   1040 VV$=INPUT$(1):VV=VAL(VV$):IFVV=0THENGOSUB930:GOSUB760:J=USR(XA):RETUR
       ELSE ONVVGOTO1100,1200,1300
IN   1100 CLS:PRINT" VIEW MODE":PRINT:PRINT" This allows quick viewing of the d
       sk sectors. The arrow keys or joystick  will alter the track/sector
       viewed.  The Spacebar or firebutton will alter  which disk is viewed
       ."
CP   1110 PRINT:PRINTB(6):PRINT:PRINTB(2):PRINT"F2 "B(3):PRINT"F3 "B(4):PRINT"F
       Go to HELP display":PRINT"F5 Set Track/Sector to 20/15"
AD   1140 GOTO1400
FJ   1200 CLS:PRINT" MOD. MODE":PRINT:PRINT" This allows you to view the conten
       s  of individual sectors in greater detailand alter them. You may use
       the select key to choose either Hex or ASC style  of input while in AL
       TER."
```

```
HG    1210 PRINT:PRINTB(6):PRINT:PRINTB(2):PRINT"F2 "B(3):PRINT"F3 "B(5):PRINT"F
        ALTER a row in the buffer":PRINT"F5 BURN the buffer to the sector"
CO    1220 PRINT"    currently displayed"
AC    1240 GOTO1400
EM    1300 CLS:PRINT" COPY MODE":PRINT:PRINT" This allows you to copy one sector
        of  a disk to another sector on either the same or different disk."
EC    1310 PRINT:PRINTB(6):PRINT:PRINTB(2):PRINT"F2 "B(5):PRINT"F3 "B(4):PRINT"F
        SeLeCT origin and destination          sectors":PRINT"F5 DUMP 1st sec
      tor to 2nd sector"
AB    1340 GOTO1400
BO    1400 LOCATE0,21:PRINT"PRESS <ENTER> TO CONTINUE";
BF    1410 VV$=INPUT$(1):IFVV$<>CHR$(13)GOTO1410ELSEGOTO1000
BG    1990 RETURN
GL    2000 DATA 0,0,0,-1,1,-1,1,0,1,1,0,1,-1,1,-1,0,-1,-1
AI    2010  DATA CD,C3,1C,E5,DD,E1,01,00
CM    2020  DATA 00,11,28,00,21,vid,00,CD
EP    2030  DATA 3C,37,DD,7E,00,C5,18,0F
DP    2040  DATA 0E,7E,B9,30,0A,0E,20,B9
EB    2050  DATA 38,03,91,18,02,C6,60,D3
GM    2060  DATA 80,C1,0C,DD,23,3E,wit,B9
CG    2070  DATA 20,06,19,CD,3C,37,0E,00
BF    2080  DATA 10,D8,C9,END
JC    2090 DATA F1 Change Display from HEX/ASC,Go to COPY mode,Go to MOD. mode,G
        to VIEW mode,Function Keys will do the following-
EF    3000 LOCATE 0,20,1:GOSUB710

END
```

This is line 930 - Type this in now and don't forget about the arrows

```
930 CLS:LOCATEPX+1,0:PRINT"DISK SCANNER":LOCATEPX+1,1:PRINT"by L.A.Dunnin
 ":LOCATE0,3:PRINT"↑↓":PRINT"←→":LOCATEPX,PY:PRINT"┌"STRING$(16,"─")"
 ┐":FORA=1TO16:LOCATEPX,PY+A,0:PRINT"│"SPACE$(16)"│";:NEXT:LOCATEPX,PY+
 17:PRINT"└"STRING$(16,"─")"┘":RETURN
```

The other characters in line 930 are standard SVI Graphics accessed using the Left & Right Graph Keys plus the appropriate other keys on the keyboard. Look them up in your handbook.